

Convolutional Networks

Presented by Ranga Rodrigo

Slides are from the sources particularly from Justin Johnson, Derek Hoiem and Svetlana Lazebnik.

Linear Classifier

stretch pixels into single column



input image

0.2	-0.5	0.1	2.0
1.5	1.3	2.1	0.0
0	0.25	0.2	-0.3

W

56
231
24
2

x_i

+

1.1
3.2
-1.2

b

→

-96.8
437.9
61.95

$f(x_i; W, b)$

cat score

dog score

ship score

plane

car

bird

cat

deer

dog

frog

horse

ship

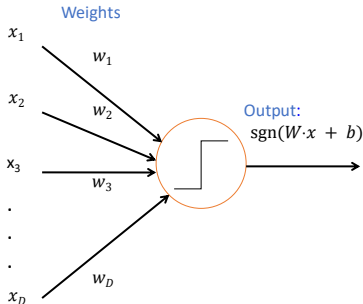
truck



Example learned weights at the end of learning for CIFAR-10. Note that, for example, the ship template contains a lot of blue pixels as expected. This template will therefore give a high score once it is matched against images of ships on the ocean with an inner product.

The Perceptron

Input



Perceptron is a linear classifier.

Capturing b also into W

$$f = \text{sgn}(Wx), x \in \mathbb{R}^D, W \in \mathbb{R}^{K \times D}$$

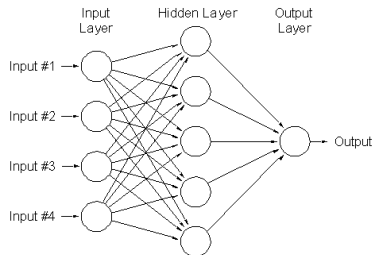
classes (pointing to K) *feature* (pointing to D)

assuming K classes.

Not powerful.

this is linear

Two-Layer Neural Network



Can learn nonlinear functions provided each perceptron has a differentiable nonlinearity.



If we add one more layer

$$f(x) = \text{sgn}(W_2 W_1 x), W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{K \times H}$$

Still

$$f(x) = \text{sgn}(W_3 x), \text{ where } W_3 = W_2 W_1$$

is a linear classifier.

To have a powerful classifiers, non-linearity at each layer is important.

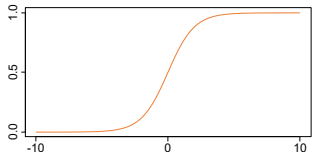
If we add non linearities, we can learn a continuous boundary.

$$f(x) = \max(0, W_2 \max(0, W_1 x))$$

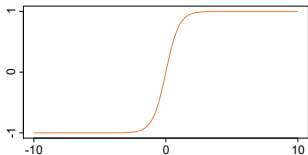
Now, neural net's first layer is bank of templates; Second layer recombines templates

Activation Functions

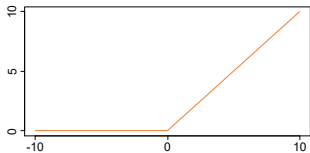
Sigmoid $\sigma(t) = \frac{1}{1 + e^{-t}}$



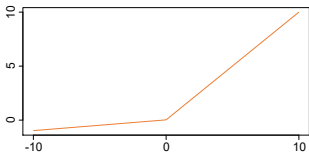
Tanh $\tanh(t)$



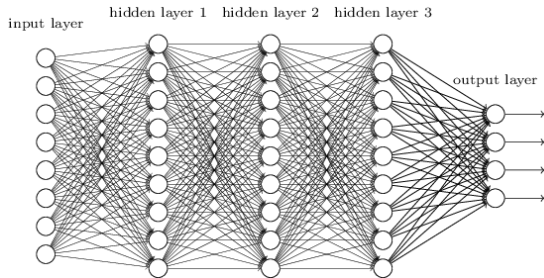
ReLU $\max(0, t)$ *good*



Leaky ReLU $\max(0.1t, t)$ *good*



Multi-Layer Neural Network



Training of Multi-Layer Networks

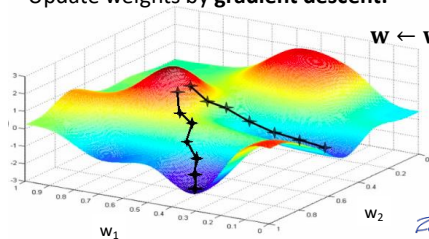
- Find network weights to minimize the *training error* between true and estimated labels of training examples, e.g.:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2 + R(W)$$

true
data
pred

no noise as ce
use all imgs in
this. not batches

- Update weights by **gradient descent**:



$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

derivative is
always size of
 $\mathbf{w}$ since L is a
scalar

but how to get grad in code?
pytorch and tensorflow do automatic
backward coded for directed
graphs for

since ce can get grad for e^*
add
 $w, x, w_1, w_2, \dots$

if there are 10 weights
 $\frac{\partial L}{\partial \mathbf{w}_n} \in \mathbb{R}^1$? Tensors

Training of Multi-Layer Networks

why noise in loss curve? Our objective function changes with every batch. So regular gradient descent is not good.

- Find network weights to minimize the *training error* between true and estimated labels of training examples, e.g.:

$$L(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y_i - f_{\mathbf{w}}(\mathbf{x}_i))^2 + R(W)$$

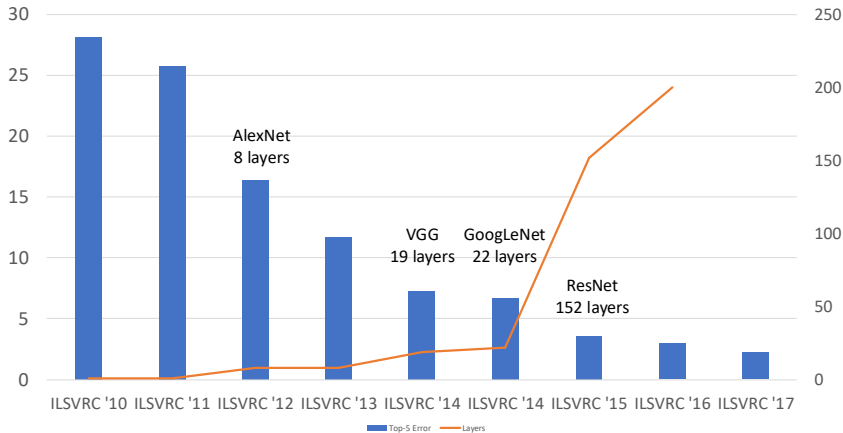
- Update weights by **gradient descent**:

$$\mathbf{w} \leftarrow \mathbf{w} - \alpha \frac{\partial L}{\partial \mathbf{w}}$$

- Back-propagation**: gradients are computed in the direction from output to input layers and combined using chain rule
- Stochastic gradient descent**: compute the weight update w.r.t. a small batch of examples at a time, cycle through training examples in random order in multiple epochs

one full cycle = epoch

Top-5 Error on ImageNet



Convolution Example: Box Filter

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[.,.]$

$h[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10							

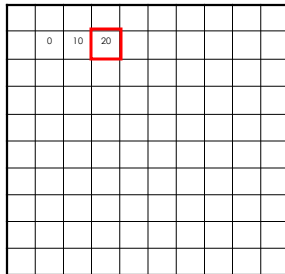
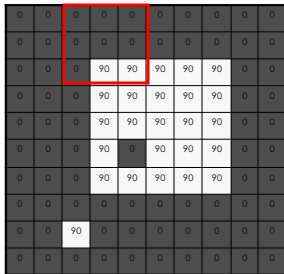
$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$



$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[\cdot, \cdot]$

$h[\cdot, \cdot]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30				
						?			
				50					

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

Image Filtering

$$g[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$f[.,.]$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$h[.,.]$

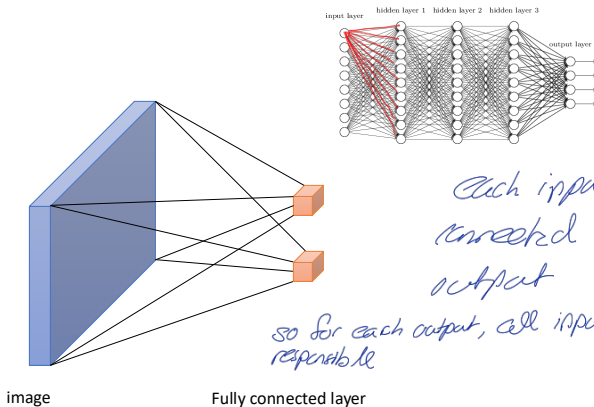
	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

*there is
a 1 pixel
gap*

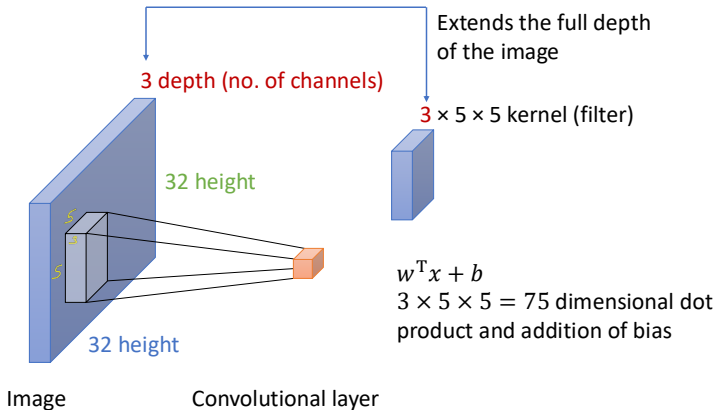
*so padding
must be done
to keep size
same*

$$h[m, n] = \sum_{k, l} g[k, l] f[m + k, n + l]$$

From Fully-Connected to Convolutional Networks

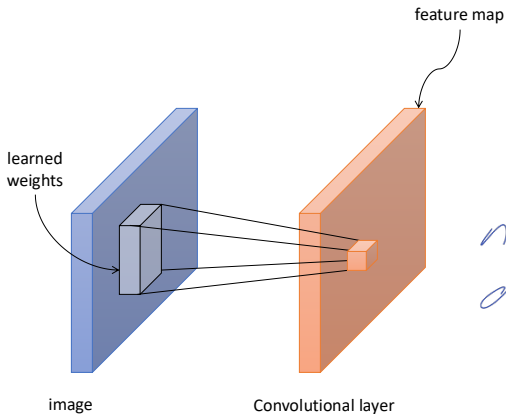


From Fully-Connected to Convolutional Networks



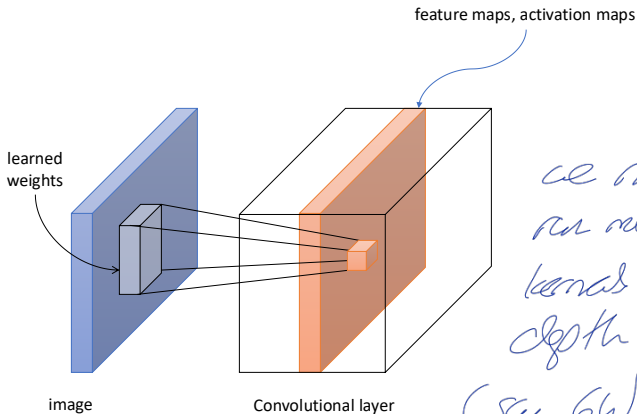
we do not talk about the depth 2. But it is there

From Fully-Connected to Convolutional Networks



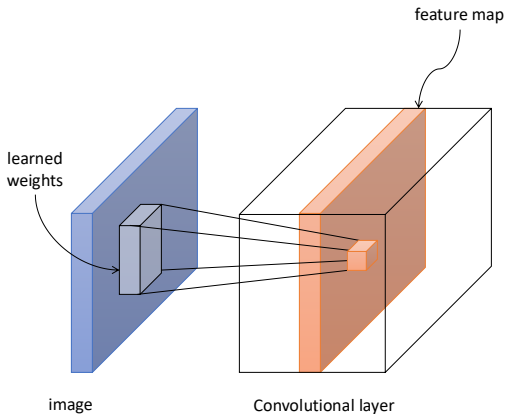
*no depth
depth = 1*

From Fully-Connected to Convolutional Networks

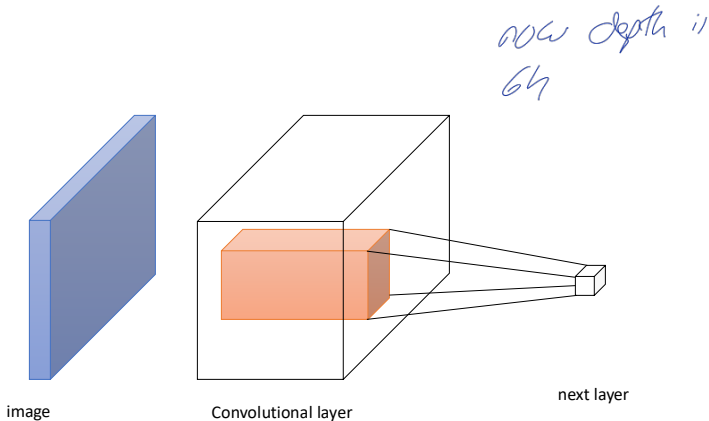


*we need to
run multiple
kernels to get
depth
(say 64)*

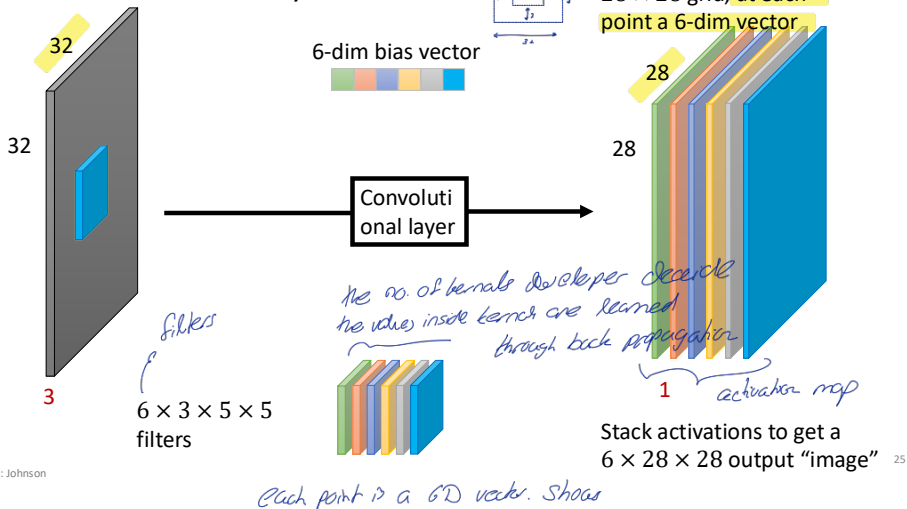
From Fully-Connected to Convolutional Networks



From Fully-Connected to Convolutional Networks

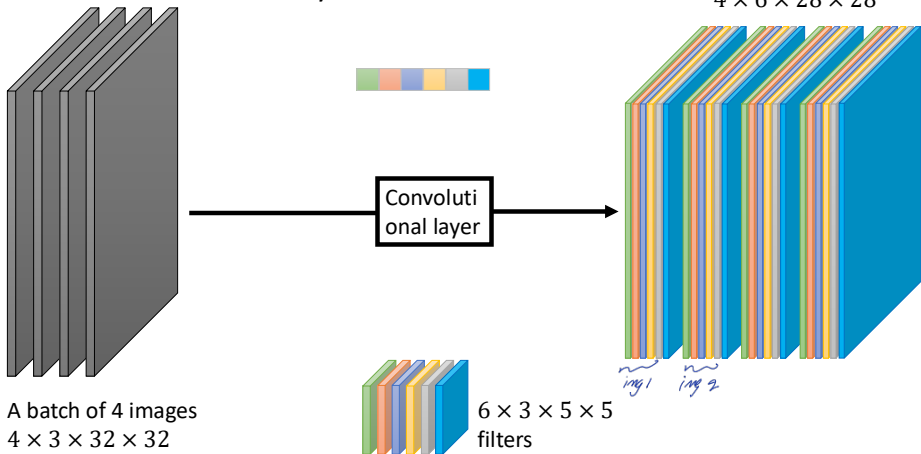


Convolutional Layer



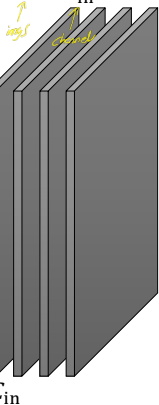
Convolutional Layer

A batch of outputs
 $4 \times 6 \times 28 \times 28$



A batch N of images

$$N \times C_{\text{in}} \times H \times W$$



Bias vector
 $C_{\text{out}}\text{-dim}$



Convoluti
onal layer

Filters

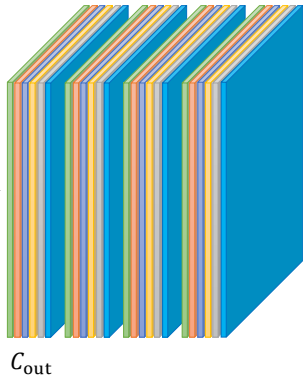
$$C_{\text{out}} \times C_{\text{in}} \times K_h \times K_w$$

↑
no of
filters



A batch of outputs

$$N \times C_{\text{out}} \times H' \times W'$$

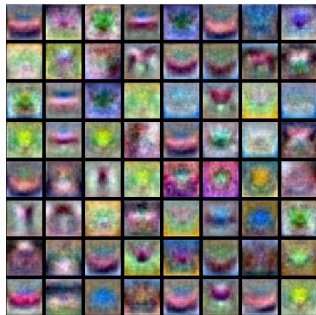


Learnt Filters



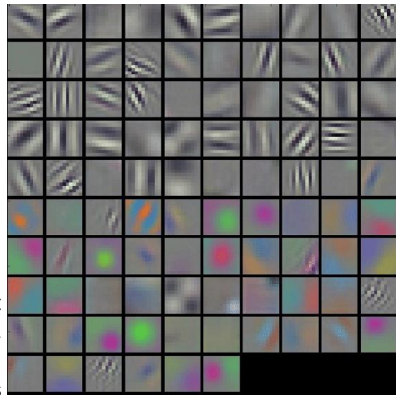
Linear classifier

<https://cs231n.github.io/linear-classify/>



Fully –
connected
From Justin
Johnson
slides

AlexNet
layer 1
 $11 \times$
11 weights

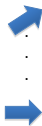


https://www.researchgate.net/figure/First-layer-filters-of-AlexNet-trained-on-ImageNet-A-considerable-amount-of-redundancy_fig4_338162455

Convolution as Feature Extraction

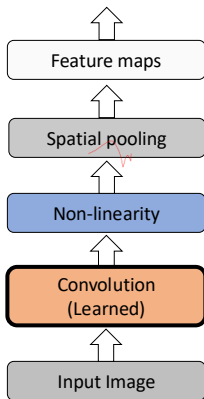


Input



Feature Map

Key Operations in a CNN



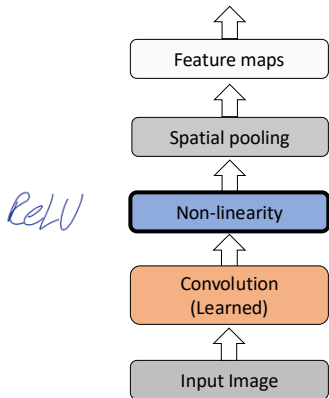
Input



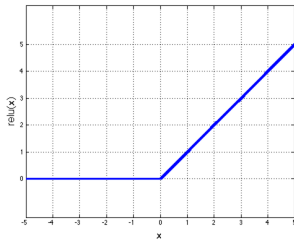
Feature Map

bag of features
There are image patches
and we make a
Vocabulary, and make
a histogram.
Then we that to determine
other images?

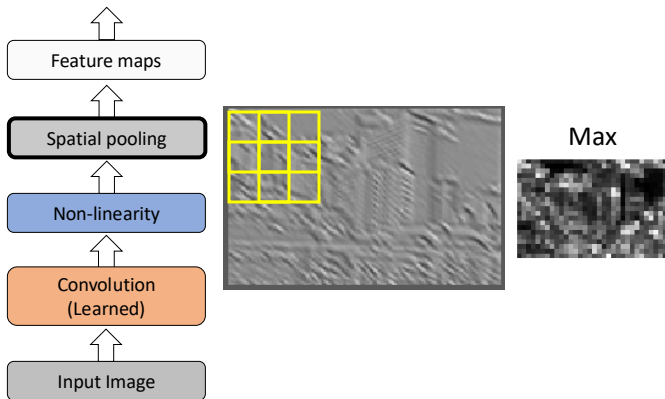
Key Operations



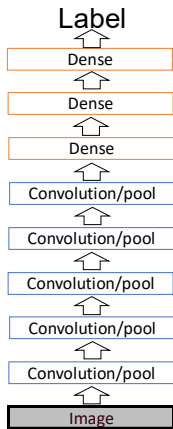
Rectified Linear Unit (ReLU)



Key Operations



Key Idea: Learn Features and Classifier That Work Well Together (“End-to-End Training”)



*This is
one layer*

Shrinking Due to Convolutions and Padding

$$\frac{1}{9}$$

1	1	1
1	1	1
1	1	1

0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	90	0	90	90	90	0	0	0
0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0

	0	10	20	30	30	30	20	10		
	0	20	40	60	60	60	40	20		
	0	30	60	90	90	90	60	30		
	0	30	50	80	80	90	60	30		
	0	30	50	80	80	90	60	30		
	0	20	30	50	50	60	40	20		
	10	20	30	30	30	30	20	10		
	10	10	10	0	0	0	0	0		

Input: 10x10

Filter: 3x3

Output: 8x8

In general:

Input: W ($W \times W$)

Filter: K ($K \times K$)

Output: $W - K + 1$

Shrinking Due to Convolutions and Padding

0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	90	0	90	90	90	0	0	0
0	0	0	0	90	90	90	90	90	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	90	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0

Input: 10x10

Filter: 3x3 $\Rightarrow$ padding must be 1

Output: 8x8

In general:

Input: W ($W \times W$)

Filter: K ($K \times K$)

Output: $W - K + 1$

Padding: P

Output: $W - K + 1 + 2P$

Set $P = (K - 1) / 2$ to

make output have same size as input.

Receptive Field

grow linearly with layers

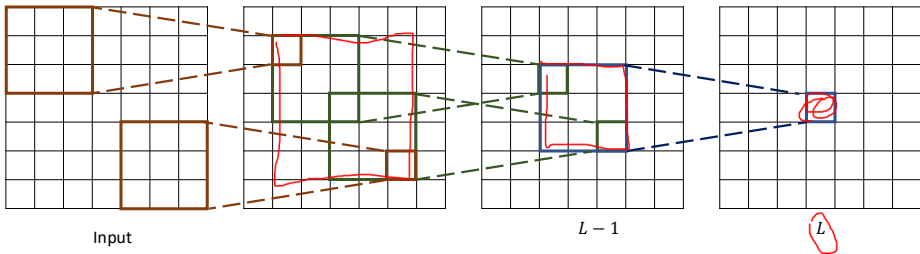
7x7

5x5

$K = 3$

3x3

1

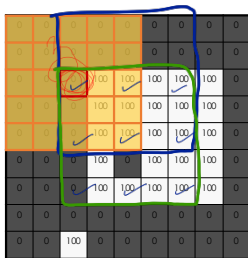


- Each successive convolution adds $K - 1$ to the receptive field size. With L layers the receptive field size is $1 + L \times (K - 1)$
- Receptive field grows linearly with layers.
- Down-sample.

why important $\Rightarrow$ 1 cell in output corresponds to how much cells in the original

Stride

move s pixels at a time



Down samples

24	64	64
36	60	

Input: 9×9

Filter: 5×5 (all $1/25$)

Stride: 2

Output: 3×3

In general:

Input: W ($W \times W$)

Filter: K ($K \times K$)

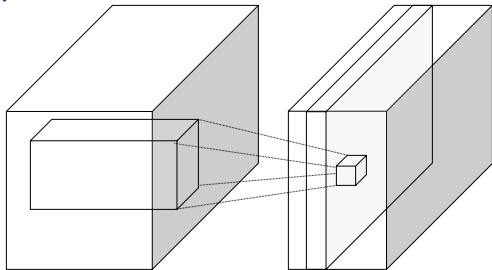
Stride: $S = 2$ in here

Output: $(W - K + 2P)/S + 1$

↑
padding?

1x1 Convolutions

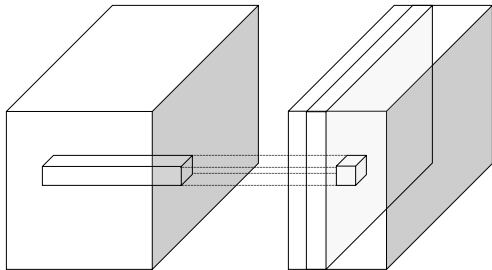
This is a special conv



conv layer

1x1 Convolutions

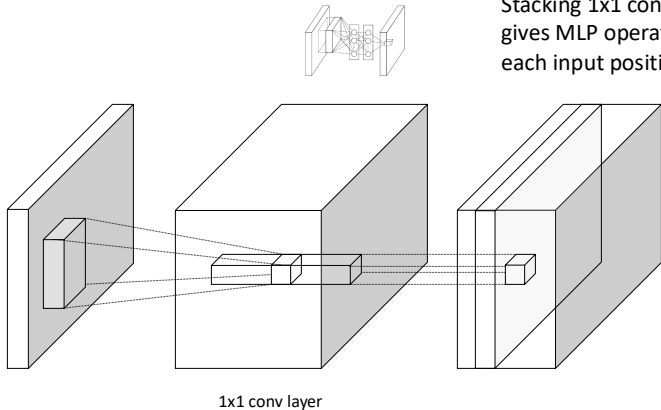
this is 1x1



1x1 conv layer

add across channels

1x1 Convolutions



Computing the No. of Learnable Parameters

If an input image is of size 32×32 grayscale, and there are 6 filters of dimension 5×5 filters, and if stride is 1, padding is 2, compute

1. Output feature map size, 2. No. of learnable parameters, 3. No. of learnable parameters if the image were a color image 4. Number of multiply-add operations for color image

Output size

$$(W - K + 2P)/S + 1 = (32 - 5 + 2 \times 2)/1 + 1 = 32 \rightarrow 6 \times 32 \times 32 = 6144$$

No. of learnable parameters

Parameters per filter = $1 \times 5 \times 5 + 1$ (+1 for the bias) = 26

Total no of learnable parameters for the 6 filters = $26 \times 6 = 156$

↑
for each pixel
we need
 $3 \times 5 \times 5 = 75$
operations.

If color

Parameters per filter = $3 \times 5 \times 5 + 1 = 76$, Total for 6 filters = $76 \times 6 = 456$

Multiply-add operations for color

Each element in output needs 75 operations. Total multiply-add = $75 \times 6144 = 460,800$

Convolution Summary

Input: $C_{\text{in}} \times H \times W$

Hyperparameters:

- **Kernel size:** $K_H \times K_W$
- **Number filters:** C_{out}
- **Padding:** P
- **Stride:** S

Weight matrix: $C_{\text{out}} \times C_{\text{in}} \times K_H \times K_W$

giving C_{out} filters of size $C_{\text{in}} \times K_H \times K_W$

Bias vector: C_{out}

Output size: $C_{\text{out}} \times H' \times W'$

where:

- $H' = (H - K + 2P)/S + 1$
- $W' = (W - K + 2P)/S + 1$

Common settings:

$K_H = K_W$ (Small square filters)

$P = (K - 1) / 2$ ("Same" padding)

$C_{\text{in}}, C_{\text{out}} = 32, 64, 128, 256$ (powers of 2)

$K = 3, P = 1, S = 1$ (3x3 conv)

$K = 5, P = 2, S = 1$ (5x5 conv)

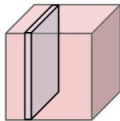
$K = 1, P = 0, S = 1$ (1x1 conv)

$K = 3, P = 1, S = 2$ (Downsample by 2)

Max Pooling

*get a 2x2
and get max*

224x224x64



Single depth slice

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Max pooling with 2×2
kernel size and stride 2



6	8
3	4

*image will slightly shift
but will not change the
output*

Introduces invariance to small spatial shifts No learnable parameters!

Max Pooling Summary

Input: $C \times H \times W$

Hyperparameters:

- Kernel size: K
- Stride: S
- Pooling function (max, avg)

Output: $C \times H' \times W'$ where

- $H' = (H - K) / S + 1$
- $W' = (W - K) / S + 1$

Learnable parameters: None!

Common settings:

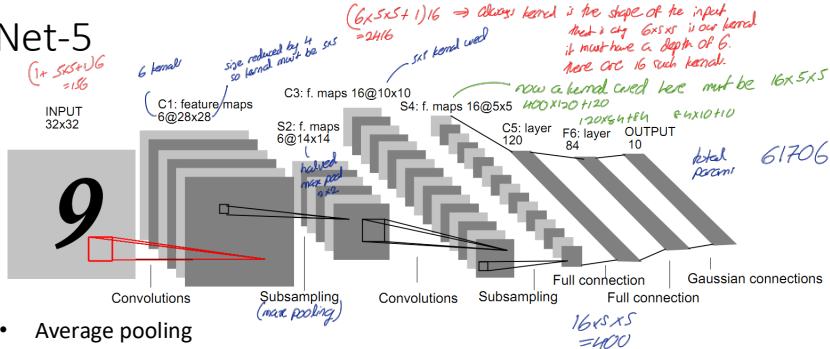
max, $K = 2, S = 2$

max, $K = 3, S = 2$ (AlexNet)

Existing Architectures

Slides are mainly from Johnson. Some slides are from Lazebnik. Some slides have been modified.

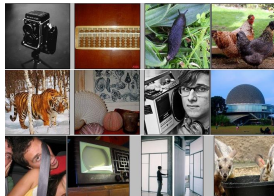
LeNet-5



- Average pooling
- Sigmoid or tanh nonlinearity
- Fully connected layers at the end
- Trained on MNIST digit dataset with 60K training examples

The Arrival of Big Visual Data...

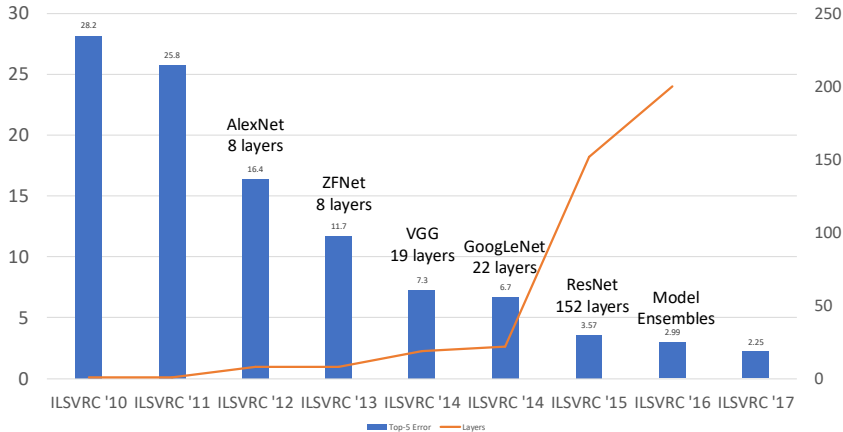
IMGENET



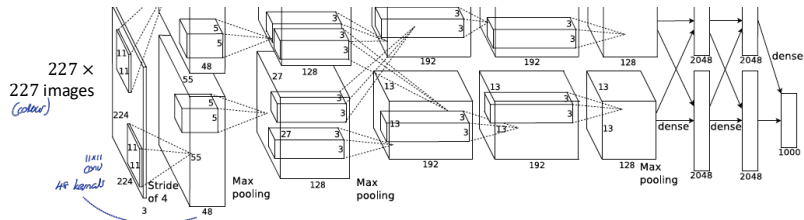
- ~14 million labeled images, 20k classes
- Images gathered from Internet
- Human labels via Amazon MTurk
- ImageNet Large-Scale Visual Recognition Challenge (ILSVRC):
1.2 million training images, 1000 classes

www.image-net.org/challenges/LSVRC/

Top-5 Error on ImageNet



AlexNet: ILSVRC 2012 Winner



Similar framework to LeNet but:

Max pooling, ReLU nonlinearity

More data and bigger model (7 hidden layers, 5 convolutional layers, 3 fully-connected layers 650K units, 60M params.)

GPU implementation (two 3GB GTX580 cards, 50x speedup over CPU)

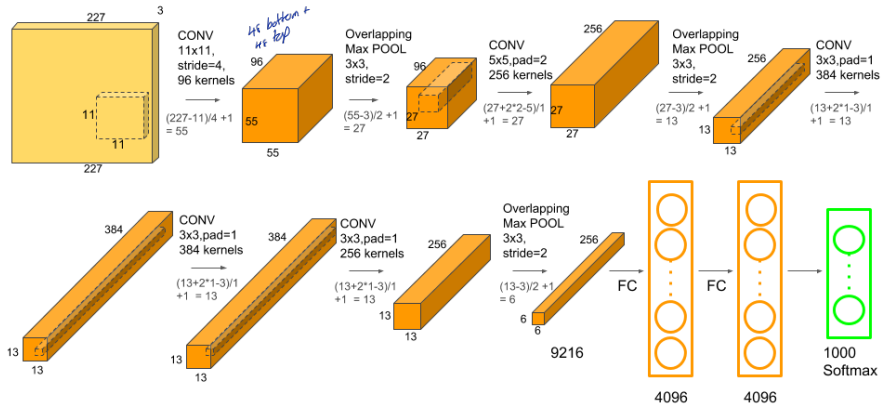
Trained on two GPUs for a week

Dropout regularization

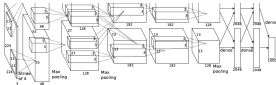
A. Krizhevsky, I. Sutskever, and G. Hinton, [ImageNet Classification with Deep Convolutional Neural Networks](#), NIPS 2012

AlexNet (Simplified)

8 layers



AlexNet



Layer	Input Size		Layer				Output size		Mem. (KB)	Param. (k)	Flops (M)
	C	H / W	Filters	Kernel	Stride	Pad		H' / W'			
	C_{in}		C_{out}	K	S	P	C_{out}				
conv1	colour 3	227	96	11	4	0	? 96	?	?	?	?

48+48

$$\begin{aligned}
 W' &= (W - K + 2P)/S + 1 \\
 &= (227 - 11 + 2 \times 0)/4 + 1 \\
 &= 55
 \end{aligned}$$

$$\begin{aligned}
 KB &= \text{No. of Elements} \times \text{Bytes per element} / 1024 \\
 &= C_{out} \times H' \times W' \times 4 / 1024 \text{ (32-bit floating-point)} \\
 &= 96 \times 55 \times 55 \times 4 / 1024 = 1134
 \end{aligned}$$

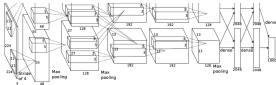
$$\begin{aligned}
 \text{Params.} &= \text{Weights} + \text{Bias} \\
 &= C_{out} \times C_{in} \times K \times K + C_{out} \\
 &= 96 \times 3 \times 11 \times 11 + 96 \\
 &= 34,944
 \end{aligned}$$

$$\begin{aligned}
 \text{No. of floating-point operations (multiply-add)} \\
 &= \text{No. of output elements} \times \text{Operations per element} \\
 &= (96 \times 55 \times 55) \times (3 \times 11 \times 11) = 105,415,200
 \end{aligned}$$

Look at output stride and determine how many operations needed per pixel

$$\begin{aligned}
 H' &= (H - K + 2P)/S + 1 \\
 &= 227 - 11
 \end{aligned}$$

AlexNet



Layer	Input Size		Layer				Output size				
	C	H / W	Filters	Kernel	Stride	Pad		H' / W'	Mem. (KB)	Param. (k)	Flops (M)
	C_{in}		C_{out}	K	S	P	C_{out}				
conv1	3	227	96	11	4	2	96	55	1134	35	105
pool1	96	55		3	2	0	?	?	?	?	?

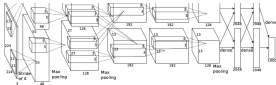
$$\begin{aligned}
 W' &= (W - K + 2P)/S + 1 \\
 &= (56 - 3 + 2 \times 0)/2 + 1 \\
 &= 27.5 \rightarrow \text{floor} \rightarrow 27
 \end{aligned}$$

Pooling layers have no learnable parameters.

$$\begin{aligned}
 \text{KB} &= \text{No. of Elements} \times \text{Bytes per element} / 1024 \\
 &= C_{out} \times H' \times W' \times 4 / 1024 \text{ (32-bit floating-point)} \\
 &= 96 \times 27 \times 27 \times 4 / 1024 = 273.375
 \end{aligned}$$

$$\begin{aligned}
 \text{No. of floating-point operations (multiply-add)} \\
 &= \text{No. of output elements} \times \text{Operations per element} \\
 &= 96 \times 27 \times 27 \times 3 \times 3 = 629,856 = 0.6 \text{ Mflops}
 \end{aligned}$$

AlexNet

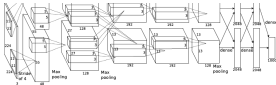


Layer	Input Size		Layer				Output size				
	C	H / W	Filters	Kernel	Stride	Pad		H' / W'	Mem. (KB)	Param. (k)	Flops (M)
	C_{in}		C_{out}	K	S	P	C_{out}				
conv1	3	227	96	11	4	2	96	55	1134	35	105
pool1	96	55		3	2	0	96	27	273	0	0
conv2	96	27	256	5	1	2	256	27	729	615	448
pool2	256	27		3	2	0	256	13	169	0	0
conv3	256	13	384	3	1	1	384	13	254	885	150
conv4	384	13	384	3	1	1	384	13	254	1,327	224
conv5	384	13	256	3	1	1	256	13	169	885	150
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0

Flatten output size

$$\begin{aligned}
 &= C_{in} \times H \times W \\
 &= 256 \times 6 \times 6 \\
 &= 9,216
 \end{aligned}$$

AlexNet



	Input Size		Layer				Output size				
Layer	C	H / W	Filters	Kernel	Stride	Pad		H' / W'	Mem. (KB)	Param. (k)	Flops (M)
	C_{in}		C_{out}	K	S	P	C_{out}				
conv1	3	227	96	11	4	2	96	55	1134	35	105
pool1	96	55		3	2	0	96	27	273	0	0
conv2	96	27	256	5	1	2	256	27	729	615	448
pool2	256	27		3	2	0	256	13	169	0	0
conv3	256	13	384	3	1	1	384	13	254	885	150
conv4	384	13	384	3	1	1	384	13	254	1,327	224
conv5	384	13	256	3	1	1	256	13	169	885	150
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,753	38

Fc6 params

$$= C_{in} \times C_{out} + C_{out}$$

$$= 9216 \times 4096 + 4096 = 37,752,832$$

Fc6 flops

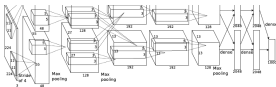
$$= C_{in} \times C_{out}$$

$$= 9216 \times 4096 = 37,748,736$$

AlexNet

kernel size and no of dannels are from

Extensive trial and error!



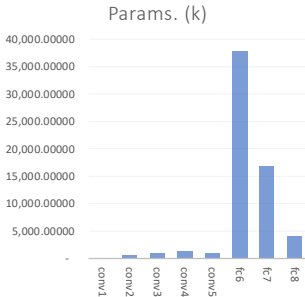
Input Size		Layer					Output size				
Layer	C	H / W	Filters	Kernel	Stride	Pad		H' / W'	Mem. (KB)	Param. (k)	Flops (M)
	C_{in}		C_{out}	K	S	P	C_{out}				
conv1	3	227	96	11	4	2	96	55	1134	35	105
pool1	96	55		3	2	0	96	27	273	0	0
conv2	96	27	256	5	1	2	256	27	729	615	448
pool2	256	27		3	2	0	256	13	169	0	0
conv3	256	13	384	3	1	1	384	13	254	885	150
conv4	384	13	384	3	1	1	384	13	254	1,327	224
conv5	384	13	256	3	1	1	256	13	169	885	150
pool5	256	13		3	2	0	256	6	36	0	0
flatten	256	6					9216		36	0	0
fc6	9216		4096				4096		16	37,753	38
fc7	4096		4096				4096		16	16,781	17
fc8	4096		1000				1000		4	4,096	4

AlexNet Memory, Parameters, and MFLOPs

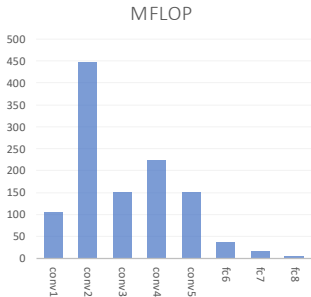
Most of the memory usage is in the early conv. layers



Nearly all parameters are in the fully-connected layers



Most floating-point ops. occur in the Conv. layers



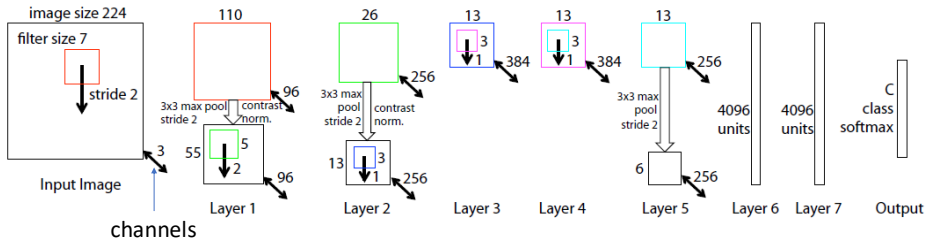
Slide: Slightly modified from Johnson

initially is to reduce Receptive size as fast as possible

fully connected layers do not have a huge computational load but have a large no. of params

ZFNet (2013 Winner, 2014) *larger version of alexnet*

ImageNet top 5 error: 16.4% -> 11.7%



Similar to AlexNet but:

CONV1: change from (11x11 stride 4) to (7x7 stride 2: down sampling only by a factor of 2)

CONV3,4,5: instead of 384, 384, 256 filters use 512, 1024, 512

More trial and error, bigger networks work better, not principle to make the networks better

Zeiler and Fergus, "Visualizing and Understanding Convolutional Networks", ECCV 2014⁵⁷

VGGNet

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels

AlexNet: 5 conv. layers

VGG has 5 conv. stages:

Stage 1: conv-conv-pool

Stage 2: conv-conv-pool

Stage 3: conv-conv-conv-[conv]-pool

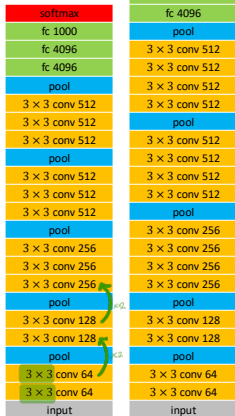
Stage 4: conv-conv-conv-[conv]-pool

Stage 5: conv-conv-conv-[conv]-pool

VGG has much more parameters, consumes much more memory, and FLOPs.



AlexNet (8)



VGG16 (16)

VGG19 (19)

Simonyan and Zissermann, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

VGGNet

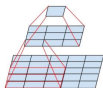
Two 3x3 convolution are equal to one 5x5 convolution but with less computational operations.

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels



Two 3×3 conv has same receptive field as a single 5×5 conv., but has fewer parameters and takes less computation!

	Option 1: One 5×5 conv.	Option 2: Two 3×3 conv.
Layer	Conv($5 \times 5, C \rightarrow C$)	Conv($3 \times 3, C \rightarrow C$) Conv($3 \times 3, C \rightarrow C$)
Params	$25C^2$ without bias	$18C^2$
FLOPs	$25C^2HW$	$18C^2HW$

softmax
fc 1000
fc 4096
fc 4096
pool
3×3 conv 256
3×3 conv 384
3×3 conv 384
pool
5×5 conv 256
pool
11×11 conv 96
input

AlexNet (8)

softmax
fc 1000
fc 4096
fc 4096
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 256
3×3 conv 256
3×3 conv 256
pool
3×3 conv 128
3×3 conv 128
pool
3×3 conv 64
3×3 conv 64
input

VGG16 (16)

softmax
fc 1000
fc 4096
fc 4096
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 512
3×3 conv 512
3×3 conv 512
pool
3×3 conv 256
3×3 conv 256
3×3 conv 256
pool
3×3 conv 128
3×3 conv 128
pool
3×3 conv 64
3×3 conv 64
input

VGG19 (19)

Simonyan and Zissermann, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

Slide: Slightly modified from Jonathon

So no reason to use 5x5

VGGNet

VGG Design rules:

All conv are 3x3 stride 1 pad 1

All max pool are 2x2 stride 2

After pool, double #channels

Input	$C \times 2H \times 2W$	$2C \times H \times W$
Layer	Conv($3 \times 3, C \rightarrow C$)	Conv($3 \times 3, 2C \rightarrow 2C$)
Mem.	$4HWC$	$2HWC$
Params	$9C^2$	$36C^2$
FLOPs	$36HWC^2$	$36HWC^2$

softmax
fc 1000
fc 4096
fc 4096
pool
3 x 3 conv 256
3 x 3 conv 384
3 x 3 conv 384
pool
5 x 5 conv 256
pool
11 x 11 conv 96
input

AlexNet (8)

softmax
fc 1000
fc 4096
fc 4096
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 256
3 x 3 conv 256
3 x 3 conv 256
pool
3 x 3 conv 128
3 x 3 conv 128
3 x 3 conv 128
pool
3 x 3 conv 64
3 x 3 conv 64
input

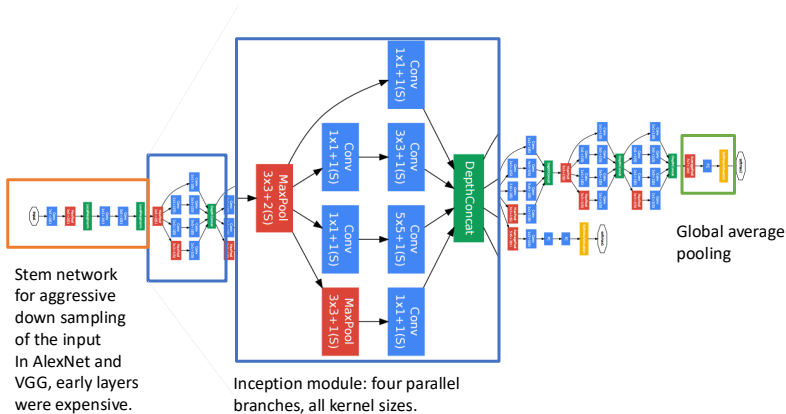
VGG16 (16)

softmax
fc 1000
fc 4096
fc 4096
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 512
3 x 3 conv 512
3 x 3 conv 512
pool
3 x 3 conv 256
3 x 3 conv 256
3 x 3 conv 256
pool
3 x 3 conv 128
3 x 3 conv 128
3 x 3 conv 128
pool
3 x 3 conv 64
3 x 3 conv 64
input

VGG19 (19)

Simonyan and Zissermann, "Very Deep Convolutional Networks for Large-Scale Image Recognition", ICLR 2015

GoogLeNet (2014):



C. Szegedy et al., [Going deeper with convolutions](#), CVPR 2015

Stem Network in GoogLeNet

	Input Size		Layer				Output size				
Layer	C	H / W	Filters	Kernel	Stride	Pad		H' / W'	Mem. (KB)	Param. (k)	Flops (M)
	C_{in}		C_{out}	K	S	P	C_{out}				
conv	3	224	64	7	2	3	64	112	3136	9	118
pool	64	112	64	3	2		64	56	784	0	2
conv	64	56	64	1	1	0	64	56	784	4	13
conv	64	56	192	3	1	1	192	56	2352	111	347
pool	192	56	192	3	2	0	192	28	588	0	1

Stem network at the start **aggressively down samples** input
(Recall in VGG-16: Most of the compute was at the start)

Total from 224 to 28 spatial resolution:	Compare VGG-16:
Memory: 7.5 MB	Memory: 42.9 MB (5.7x)
Params: 124K	Params: 1.1M (8.9x)
MFLOP: 418	MFLOP: 7485 (17.8x)

GoogLeNet Inception Module

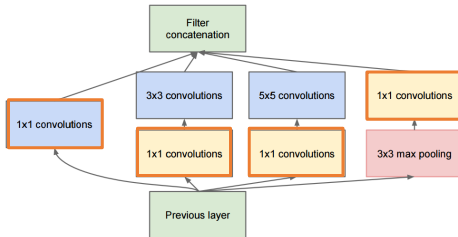
not inp?

Inception module

Local unit with
parallel branches
Local structure repeated
many times throughout the
Network

Uses 1x1 “Bottleneck”

layers to reduce channel
dimension before
expensive conv (we will
revisit this with ResNet!)



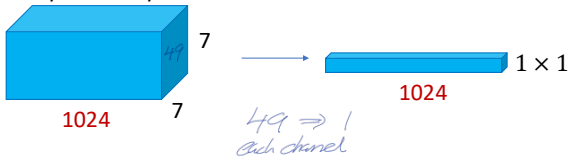
GoogLeNet: Global Average Pooling

No large FC layers at the end! Instead uses global average pooling to collapse spatial dimensions, and one linear layer to produce class scores

(Recall VGG-16: Most parameters were in the FC layers!)

Layer	Input Size		Layer				Output size		Mem. (KB)	Param. (k)	Flops (M)
	C	H / W	Filters	Kernel	Stride	Pad		H' / W'			
	C_{in}		C_{out}	K	S	P	C_{out}				
avg-pool	1024	7		7	1	0	1024	1	4	0	0
fc	1024		1000				1000		0	1025	1

Special information destroyed not by flattening, but by global average pooling. Then only one fc layer.

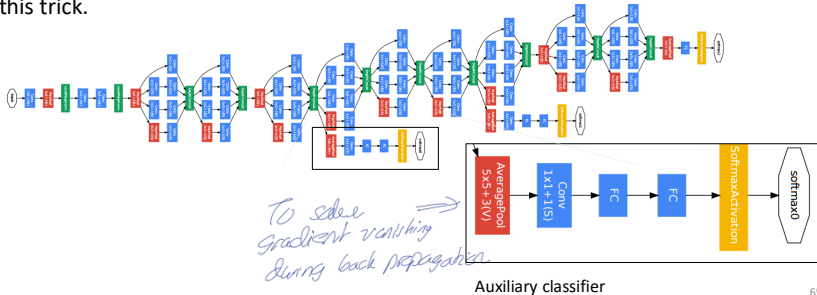


GoogLeNet: Auxiliary Classifiers

Training using loss at the end of the network didn't work well: Network is too deep, gradients don't propagate cleanly.

As a hack, attach "auxiliary classifiers" at several intermediate points in the network that also try to classify the image and receive loss

GoogLeNet was before batch normalization! With BatchNorm no longer need to use this trick.



Summary

- ImageNet, ILSVRC
- AlexNet: max pooling, ReLU nonlinearity, large no. of parameters
- VGG: all conv are 3x3 stride 1 pad 1, all max pool are 2x2 stride 2, after pool, double #channels
- GoogLeNet: stem network, inception module, global average pooling, auxiliary classifiers

Residual Networks (ResNet): 2015

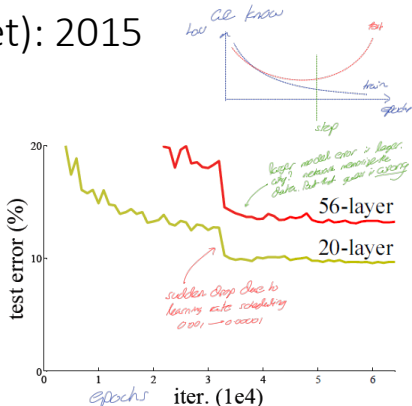
With batch normalization deeper models could be trained.

Deeper model does worse than shallow model!

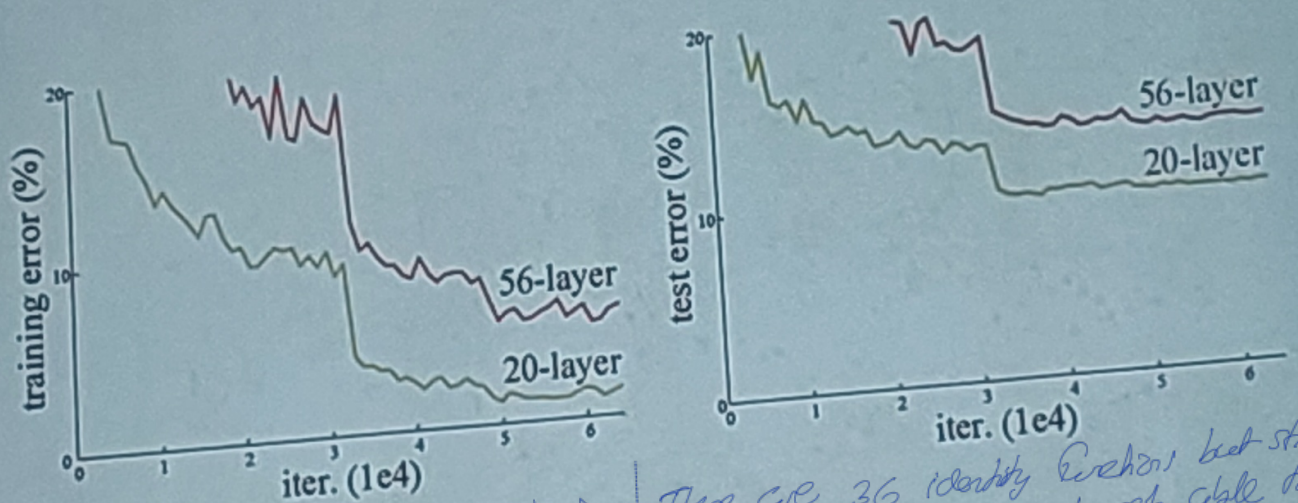
Biggert not better, in contract to previous experience.

Initial guess: Deep model is **overfitting** since it is much bigger than the other model

In fact the deep model seems to be underfitting since it also performs worse than the shallow model on the training set! It is actually underfitting. 56-layer network has not been able to emulate the 20-layer network.



Residual Networks (ResNet): 2015

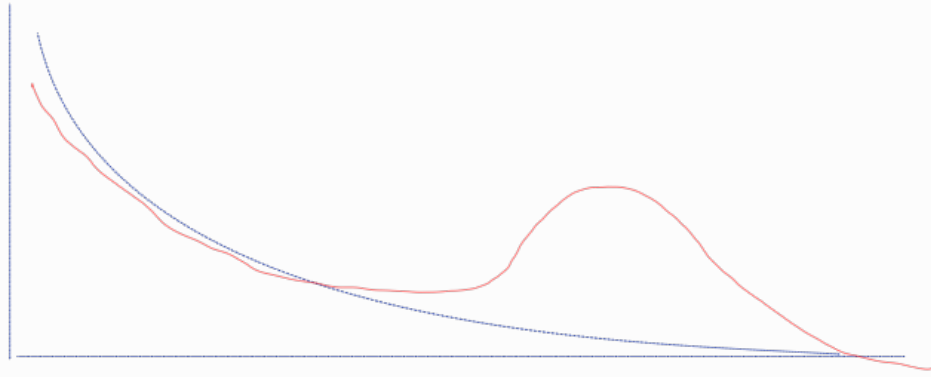


But if memorizing is done, why is the training error is high?

There are 36 identity functions, but still high conclusion: network not able to learn identity function

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

Something about Double descent
when no. of params $\gg$ data points



when the dataset is large we can reduce
the no. of training epochs to get
the desired level of accuracy.

ResNets

A deeper model can **emulate** a shallower model: copy layers from shallower model, set extra layers to identity.

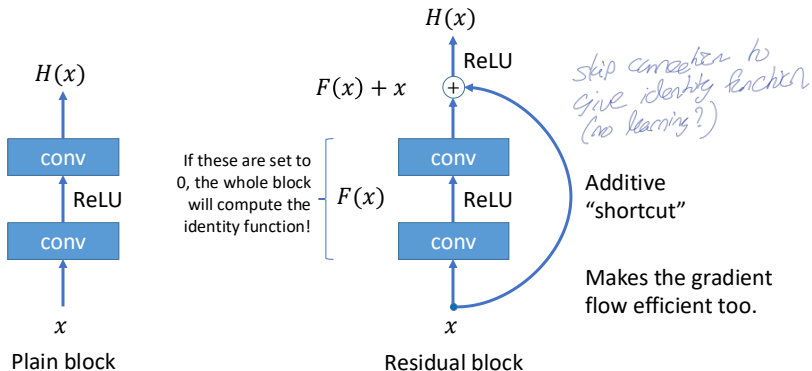
Thus deeper models should do at least as good as shallow models.

Hypothesis: This is an **optimization** problem. Deeper models are harder to optimize, and in particular don't learn identity functions to emulate shallow models.

Solution: Change the network so learning identity functions with extra layers is easy!

Residual Networks

Solution: Change the network so learning identity functions with extra layers is easy!



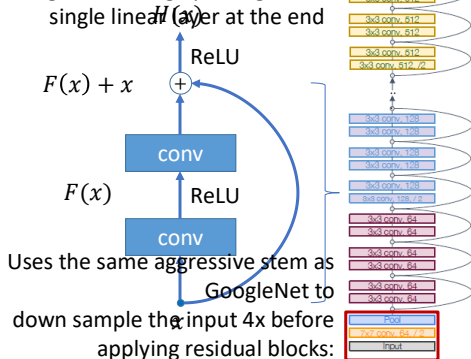
Residual Networks

A residual network is a stack of many residual blocks

Regular design, like VGG: each residual block has two 3x3 conv

Network is divided into stages: the first block of each stage halves the resolution (with stride-2 conv) and doubles the number of channels

Like GoogLeNet, no big fully-connected-layers: instead use global average pooling and a single linear layer at the end



Residual Networks

ResNet-18:

Stem: 1 conv layer

Stage 1 (C=64): 2 res. block = 4 conv

Stage 2 (C=128): 2 res. block = 4 conv

Stage 3 (C=256): 2 res. block = 4 conv

Stage 4 (C=512): 2 res. block = 4 conv

Linear

ImageNet top-5 error: 10.92

GFLOP: 1.8

ResNet-34:

Stem: 1 conv layer

Stage 1: 3 res. block = 6 conv

Stage 2: 4 res. block = 8 conv

Stage 3: 6 res. block = 12 conv

Stage 4: 3 res. block = 6 conv

Linear

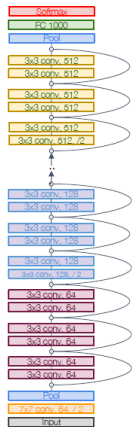
ImageNet top-5 error: 8.58

GFLOP: 3.6

VGG-16:

ImageNet top-5 error: 9.62

GFLOP: 13.6



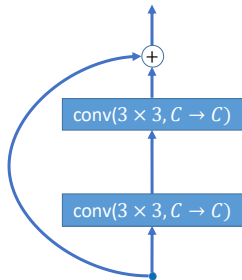
He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

Error rates are 224x224 single-crop testing, reported by torchvision

Slide: Slightly modified from Johnson

Residual Networks: Bottleneck Block

Reduces the number of channels before computing a 3×3 convolution. Deeper networks without increasing the computational cost.



More layers, less computational cost!

FLOPs: $9HWC^2$

*output feature
map size
(C x H x W) x (C x 3 x 3)*

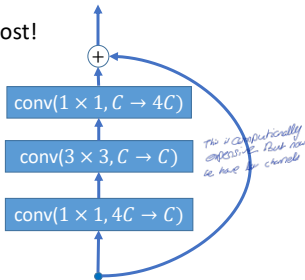
Basic residual block Total FLOPs:
 $18HWC^2$

FLOPs: $4HWC^2$

FLOPs: $9HWC^2$

FLOPs: $4HWC^2$

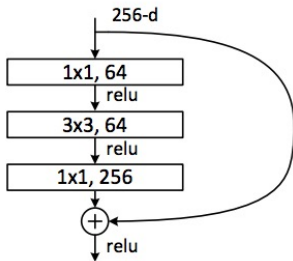
Total FLOPs:
 $17HWC^2$



Bottleneck residual block

Residual Networks

Deeper residual module
(bottleneck)



- Directly performing 3x3 convolutions with 256 feature maps at input and output:
 $256 \times 256 \times 3 \times 3 \sim 600K$ operations
- Using 1x1 convolutions to reduce 256 to 64 feature maps, followed by 3x3 convolutions, followed by 1x1 convolutions to expand back to 256 maps:
 $256 \times 64 \times 1 \times 1 \sim 16K$
 $64 \times 64 \times 3 \times 3 \sim 36K$
 $64 \times 256 \times 1 \times 1 \sim 16K$
Total: $\sim 70K$

(reduced?)

Residual Networks

			Stage 1		Stage 2		Stage 3		Stage 4				
	Block type	Stem layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	Blocks	Layers	FC layers	GFLOP	ImageNet top-5 error
ResNet-18	Basic	1	2	4	2	4	2	4	2	4	1	1.8	10.92
ResNet-34	Basic	1	3	6	4	8	6	12	3	6	1	3.6	8.58

Deeper ResNet-101 and ResNet-152 models are more accurate, but also more computationally heavy

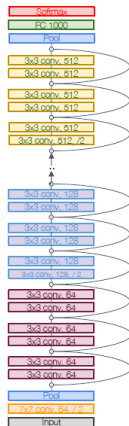
Able to train very deep networks

- Deeper networks do better than shallow networks (as expected)
- Swept 1st place in all ILSVRC and COCO 2015 competitions
- Still widely used today!

He et al, "Deep Residual Learning for Image Recognition", CVPR 2016

Error rates are 224x224 single-crop testing, reported by torchvision

Slide: Slightly modified from Johnson



ResNet: Going Real Deep

Revolution of Depth

AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, **152 layers**
(ILSVRC 2015)

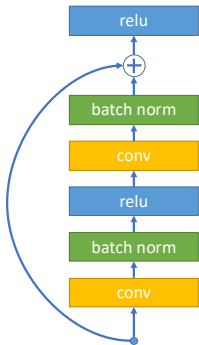


Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, [Deep Residual Learning for Image Recognition](#), CVPR 2016

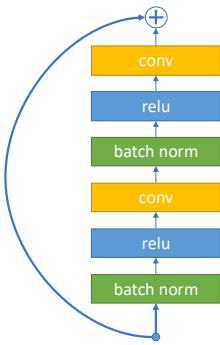
Improving Residual Networks: Block Design



Original ResNet block



"Pre-Activation" ResNet Block



Slight improvement in accuracy
(ImageNet top-1 error)

ResNet-152: 21.3 vs 21.1

ResNet-200: 21.8 vs 20.7

Not actually used that much in
practice

Bigger Not Better

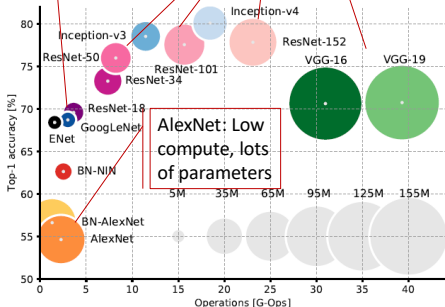
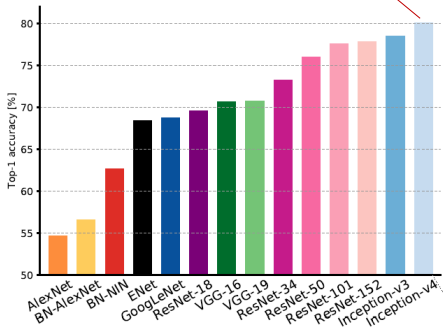
Innovations typically reduce parameters, despite deeper nets.

Inception-v4: Resnet + Inception!

GoogLeNet:
Very efficient!

VGG: Highest
memory, most
operations

ResNet: Simple design,
moderate efficiency,
high accuracy



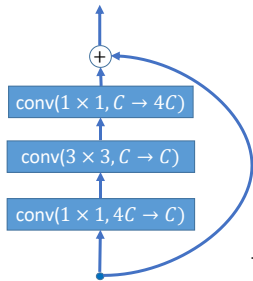
ImageNet 2016 winner: Model Ensembles



Multi-scale ensemble of Inception, Inception-Resnet, Resnet, Wide Resnet models

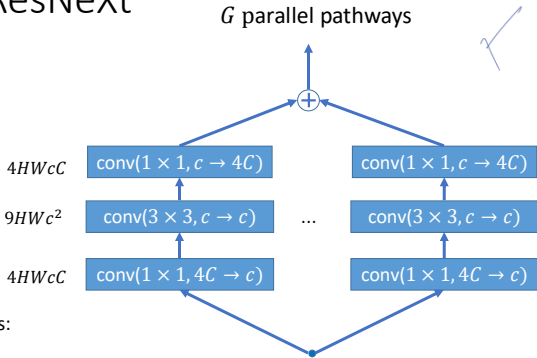
	Inception -v3	Inception -v3	Inception- ResNet-v2	ResNet-200	Wrn-68-3	Fusion (val.)	Fusion (test)
Err. (%)	4.20	4.01	3.52	4.26	4.65	2.92 (-0.6)	2.99

Improving ResNet: ResNeXt



Total FLOPs:
 $17HWC^2$

Bottleneck residual block



Total FLOPs:

$$G(4HWCc + 9HWC^2 + 4HWCc)$$

$$G(4HWCc + 9HWC^2 + 4HWCc) = 17HWC^2$$

Example: $C = 64, G = 4, c = 24$; $C = 64, G = 32, c = 4$

Xie et al, "Aggregated residual transformations for deep neural networks", CVPR 2017

Slide: Slightly modified from Johnson

Group Convolutions

Convolution with groups=1:

Normal convolution

Input: $C_{in} \times H \times W$

Weight: $C_{out} \times C_{in} \times K \times K$

Output: $C_{out} \times H' \times W'$

FLOPs: $C_{out} \times C_{in} \times K^2 \times H \times W$

All convolutional kernels touch all C_{in} channels of the input

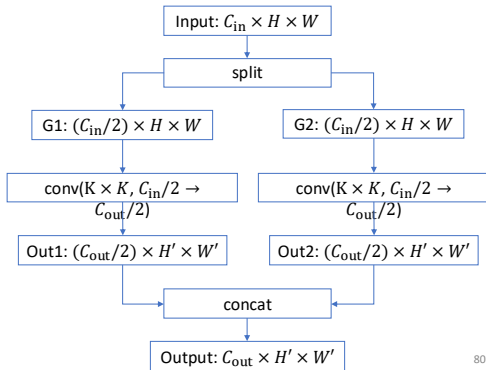
Depthwise Convolutions

Special case: $G = C_{in}$, $C_{out} = nC_{in}$

Each input channel is convolved with n different $K \times K$ filters to produce n output channels

Convolution with groups=2:

Two parallel convolution layers that work on half the channels



Group Convolutions



Convolution with groups=1:

Normal convolution

Input: $C_{in} \times H \times W$

Weight: $C_{out} \times C_{in} \times K \times K$

Output: $C_{out} \times H' \times W'$

FLOPs: $C_{out} \times C_{in} \times K^2 \times H \times W$

All convolutional kernels touch
all C_{in} channels of the input

Depthwise Convolutions

Special case: $G = C_{in}$, $C_{out} = nC_{in}$

Each input channel is convolved
with n different $K \times K$ filters to
produce n output channels

Convolution with groups= G :

G parallel conv layers; each "sees"

C_{in}/G input channels and produces

C_{out}/G output channels

Input: $C_{in} \times H \times W$

Split to $G \times [(C_{in}/G) \times H \times W]$

Weight: $G \times (C_{out}/G) \times (C_{in} \times G) \times K \times K$

G parallel convolutions

Output: $G \times [(C_{out}/G) \times H' \times W']$

Concat to $C_{out} \times H' \times W'$

FLOPs: $C_{out} \times C_{in} \times K^2 \times H \times W$

ResNeXt: Maintain Computation by Adding Groups

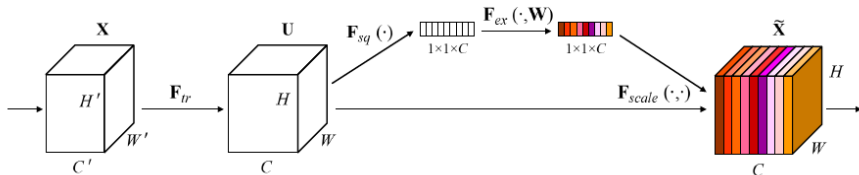


Model	Groups	Group width	Top-1 Error
ResNet-50	1	64	23.9
ResNeXt-50	2	40	23
ResNeXt-50	4	24	22.6
ResNeXt-50	8	14	22.3
ResNeXt-50	32	4	22.2

Model	Groups	Group width	Top-1 Error
ResNet-101	1	64	22.0
ResNeXt-101	2	40	21.7
ResNeXt-101	4	24	21.4
ResNeXt-101	8	14	21.3
ResNeXt-101	32	4	21.2

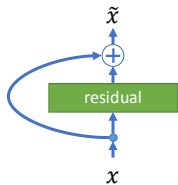
Adding groups improves performance with same computational complexity!

Squeeze-and-Excitation Networks (SENet, 2017) ✕



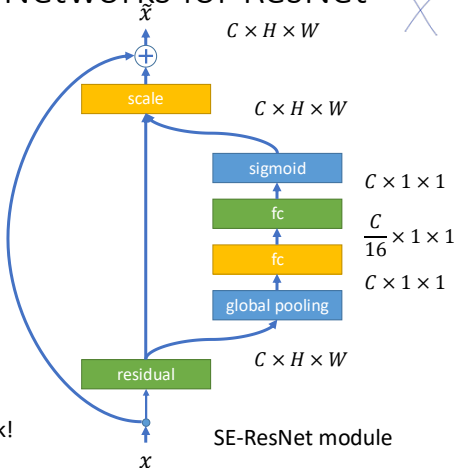
Adaptively recalibrates channel-wise feature responses by explicitly modelling interdependencies between channels.

Squeeze-and-Excitation Networks for ResNet



ResNet module

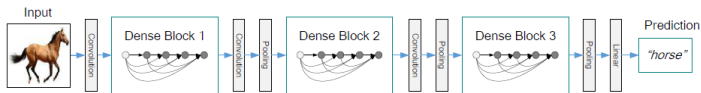
Adds a “Squeeze-and-excite” branch to each residual block that performs global pooling, full-connected layers, and multiplies back onto feature map
Adds global context to each residual block!
Won ILSVRC 2017 with ResNeXt-152-SE



SE-ResNet module

Hu et al, “Squeeze-and-Excitation networks”, CVPR 2018

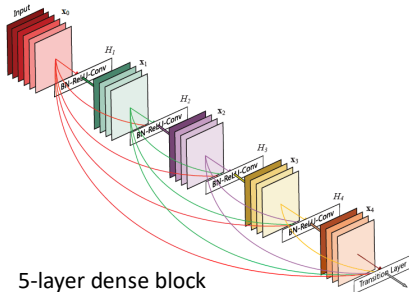
Densely Connected Neural Networks



Connects each layer to every other layer in a feed-forward fashion. Concatenation shortcuts, instead of additive.

traditional convolutional networks with L layers have L connections: densely connected network has $L(L + 1)/2$ direct connections.

Alleviates vanishing gradient, strengthens feature propagation, encourages feature reuse

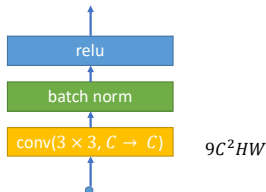


5-layer dense block

MobileNets: Tiny Networks (for Mobile Devices)

Standard Convolution Block

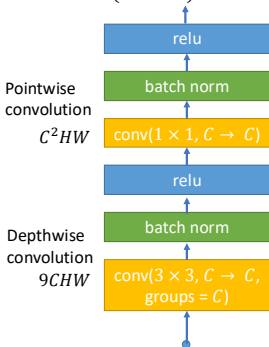
Total cost: $9C^2HW$



Speed up:
$$= 9C^2HW / (9C + C^2)HW$$
$$= 9 \text{ (as } C \rightarrow \infty)$$

Depthwise Separable Convolution

Total cost: $(9C + C^2)HW$



ShuffleNet:

Zhang et al, CVPR 2018

MobileNetV2:

Sandler et al, CVPR 2018

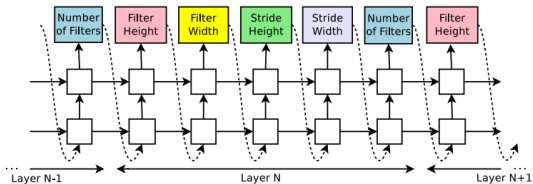
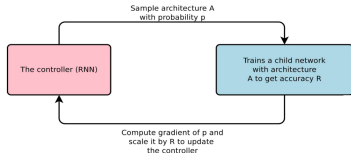
ShuffleNetV2:

Ma et al, ECCV 2018

Neural Architecture Search

Configuration string:

["Filter width: 5", "Filter height: 3", "Num filters: 24"]



Designing neural network architectures is hard – let's automate it!

- One network (controller) outputs network architectures
- Sample child networks from controller and train them
- After training a batch of child networks, make a gradient step on controller network (Using policy gradient)
- Over time, controller learns to output good architectures!

800 GPUs concurrently for 28 days
12,800 child models

Zoph and Le, "Neural Architecture Search with Reinforcement Learning", ICLR 2017

CNN Architectures Summary



- Early work (AlexNet -> ZFNet -> VGG) shows that **bigger networks work better**
- GoogLeNet one of the first to focus on **efficiency** (aggressive stem, 1x1 bottleneck convolutions, global avg pool instead of FC layers)
- ResNet showed us how to train extremely deep networks – limited only by GPU memory! Started to show diminishing returns as networks got bigger
- After ResNet: Efficient networks became central: how can we improve the accuracy without increasing the complexity?
- Lots of **tiny networks** aimed at mobile devices: MobileNet, ShuffleNet, etc.
- Neural Architecture Search promises to automate architecture design

Resnet-50 is good

EfficientNet

(network architecture)
learn

X

no guessing
on the

- Their model scaling study identified that carefully balancing network depth, width, and resolution can lead to better performance.
- Then, they use neural architecture search to design a new baseline network and scale it up to obtain a family of models, called EfficientNets.
- Depth (d): Scaling network depth is the most common way used by many ConvNets.
- Width (w): Scaling network width (no. of channels) is commonly used for small size models.
- Resolution (r): With higher resolution input images, ConvNets can potentially capture more fine-grained patterns.
- As $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, for any new ϕ , the total FLOPS will approximately increase by 2^ϕ .

$$\text{depth: } d = \alpha^\phi$$

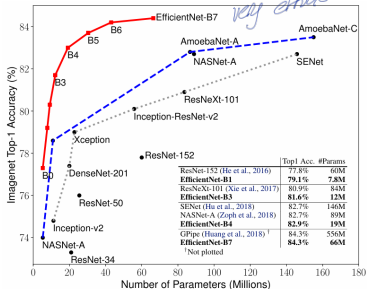
$$\text{width: } w = \beta^\phi$$

$$\text{resolution: } r = \gamma^\phi$$

$$\text{s.t. } \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$$

$$\alpha \geq 1, \beta \geq 1, \gamma \geq 1$$

very efficient



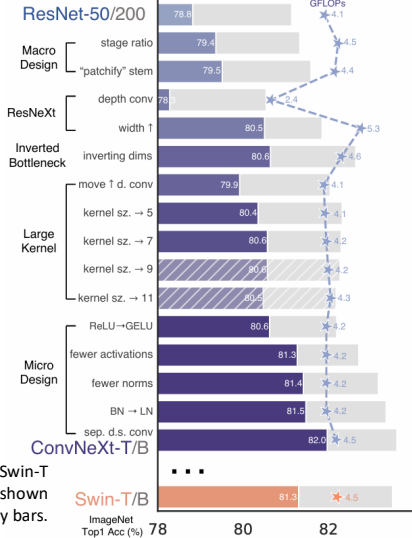
Model Size vs. ImageNet Accuracy.

ConvNeXt

large model that works as good as transformers

- The paper reexamines the design spaces and test the limits of what a pure ConvNet can achieve.
- It modernizes a standard ResNet toward the design of a vision Transformer, and discover several key components that contribute to the performance difference along the way.

The foreground bars are model accuracies in the ResNet-50/Swin-T FLOP regime; results for the ResNet-200/Swin-B regime are shown with the gray bars.



What Architecture to Use

- Don't be a hero. For most problems you should use an off-the-shelf architecture; don't try to design your own!
- If you care about accuracy, **ResNet-50**, **ResNet-101**, **EfficientNets (B3–B7)**, or **ConvNeXt** are excellent choices. EfficientNets combine high accuracy with efficiency through compound scaling, and ConvNeXt modernizes CNNs to rival Transformers.
- If you want a lightweight, efficient network (e.g., for mobile or real-time applications), consider **MobileNets**, **ShuffleNets**, or **EfficientNets (B0–B2)**.